

PLATFORM ENGINEERING · SITE RELIABILITY

Enterprise Observability & Monitoring on Kubernetes

Implementation Report — Architecture, GitOps Delivery Model, and End-to-End Metrics, Logs & Traces Configuration

Prepared by: Siddhartha Sanapala · siddhartha.sanapala@sailssoftware.com

Date: 15 July 2026

Reference implementation: Minikube · Packaged for: Google Kubernetes Engine (GKE), Amazon EKS

Table of Contents

1. Executive Summary

2. Objectives and Scope

3. System Architecture

- Architecture Diagram

- Component Inventory

- Key Architectural Decisions

4. Technology Stack and Dependencies

5. GitOps Delivery Model — ArgoCD

- Why GitOps

- The App-of-Apps Pattern

- How This Improves Kubernetes Maintainability

- Self-Healing in Practice

6. Monitoring and Observability Configuration

- Metrics — Prometheus

- Visualization — Grafana

- Logging — Loki and Promtail

- Distributed Tracing — OpenTelemetry and Tempo

- Alerting — Alertmanager and SLO Rules

- Cross-Pillar Correlation

- The Instrumented Reference Workload

7. Security and Governance

8. Phased Implementation Process

9. Multi-Cloud Packaging — GKE and EKS

10. Quality Assurance and Validation

11. Operational Lessons and Runbook

12. Conclusion and Recommendations

13. Appendix

1. Executive Summary

This report documents the design, build, and validation of a production-shaped observability platform on Kubernetes, covering the three pillars of observability — **metrics, logs, and distributed traces** — together with dashboards, automated alerting, role-based access control, network segmentation, and a real instrumented workload used to prove the pipeline end-to-end rather than with synthetic test data.

The platform was built hands-on on a locally provisioned Kubernetes cluster (Minikube, sized and configured to mirror a real node pool) and is **100% managed through ArgoCD**, meaning the Git commit history of this repository is the authoritative change history of the cluster. No component was ever deployed by a one-off manual command that isn't also reproducible from source control. The same design has been packaged as two cloud-native Helm charts — one for Google Kubernetes Engine (GKE), one for Amazon EKS — so the identical architecture can be installed on either managed cloud platform with only cloud-specific values (storage class, ingress controller, identity federation) changed.

Dimension	Delivered
Metrics	Prometheus (2 replicas), kube-state-metrics, node-exporter — 21+ scrape targets, all healthy
Logs	Loki + Promtail — cluster-wide log aggregation from every namespace
Traces	OpenTelemetry Collector + Tempo — distributed request tracing with real multi-span traces
Dashboards & Alerts	Grafana dashboards and Prometheus alert rules defined and versioned as code
Alerting	Alertmanager, 3-replica HA cluster, SLO burn-rate and infrastructure health rules
Change management	ArgoCD App-of-Apps — 14 Applications, fully GitOps-managed, self-healing
Security baseline	Namespace multi-tenancy, least-privilege RBAC, default-deny NetworkPolicy, restricted Pod Security Standards
Cloud portability	Two validated Helm charts (GKE, EKS) — linted, schema-validated, and dry-run tested against a live API server
Validation	26-check automated end-to-end gate, reproducible via a single script

Outcome: a fully reproducible, GitOps-governed observability platform that can be torn down and rebuilt from a single script, deployed identically to two major public clouds via Helm, and that has been proven — not assumed — to correlate a single user request across metrics, logs, and traces.

2. Objectives and Scope

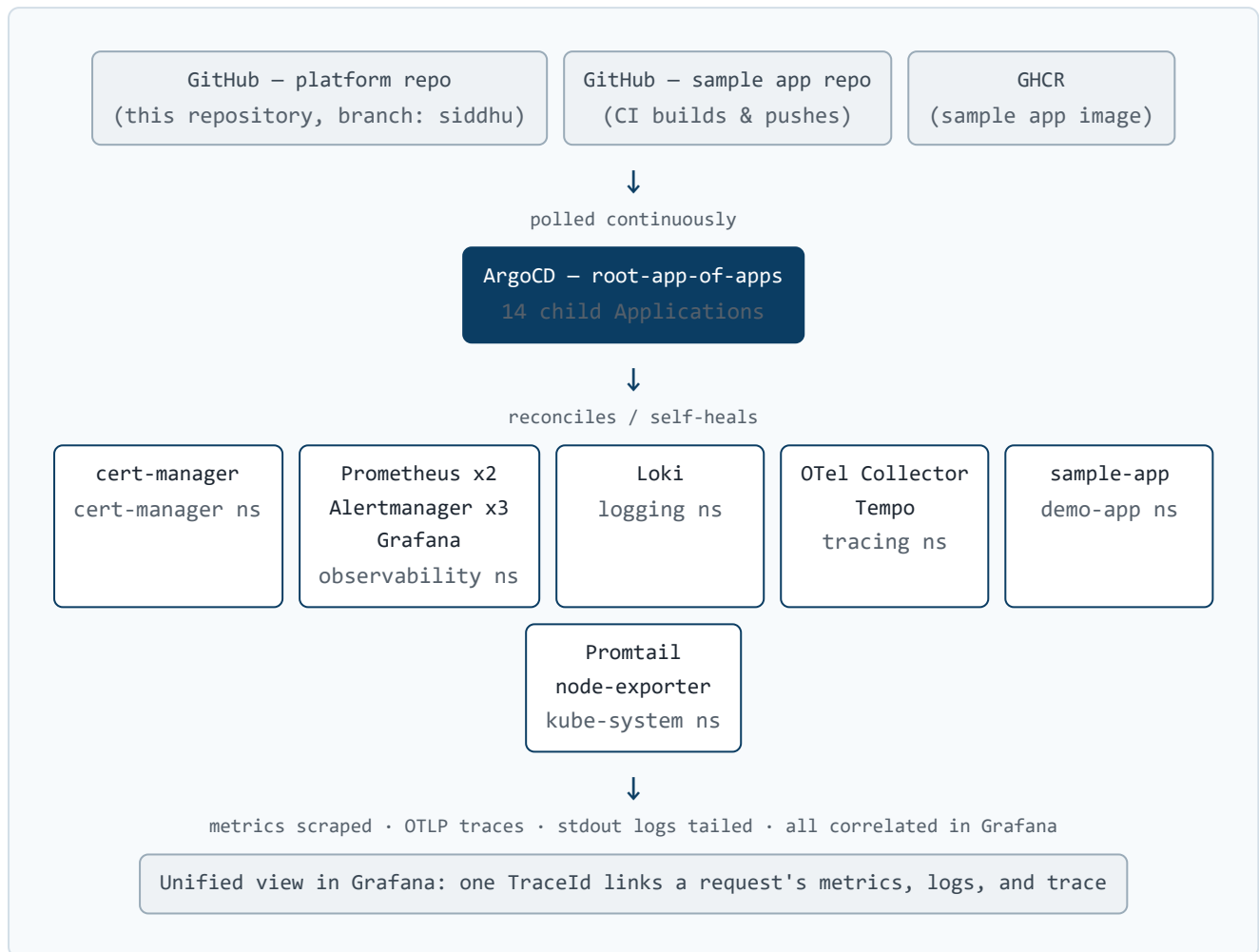
The initiative set out to answer a concrete engineering question: *what does an enterprise-grade observability platform on Kubernetes actually require, beyond installing Prometheus and Grafana and calling it done?* The scope was defined as:

1. **Full observability coverage** — not metrics alone, but metrics, logs, and traces working together, with a real workload proving correlation between them.
2. **GitOps as the operating model**, not an add-on — every piece of cluster state, from namespaces to alert rules, deployed and changed only through version-controlled declarations, reconciled automatically by ArgoCD.
3. **Enterprise baseline hygiene** — multi-tenant namespace isolation, least-privilege RBAC, network segmentation, and pod security hardening, applied from the start rather than retrofitted.
4. **Cloud portability** — the same architecture packaged so it can be installed on GKE or EKS without redesign, only cloud-specific configuration substitution.
5. **Provable correctness** — an automated validation gate that checks the platform end-to-end, so "it's done" is a test result, not an opinion.

Out of scope for this phase: production-grade identity federation (OIDC/IdP integration is designed for but not wired to a real identity provider), long-term object storage backends for Loki/Tempo (designed as an opt-in overlay, not enabled by default), and a CI/CD pipeline for this repository itself. These are called out explicitly in Section 12.

3. System Architecture

3.1 Architecture Diagram



Every arrow above except the top "polled continuously" edge represents in-cluster, NetworkPolicy-governed traffic. The operator's own `kubectl` access is limited to two one-time bootstrap commands (Section 5); all other change flows through Git.

3.2 Component Inventory

Component	Namespace	Source	Role
ArgoCD	argocd	Vendored install manifest	GitOps controller — the only component ever applied by hand
cert-manager	cert-manager	Vendored install manifest	Certificate lifecycle; self-signed issuer on Minikube, cloud-managed certs on GKE/EKS
kube-prometheus-stack	observability	Helm — prometheus-community	Prometheus, Alertmanager, Grafana, kube-state-metrics, the Prometheus Operator
Loki + Promtail	logging / kube-system	Helm — grafana	Cluster-wide log aggregation
Tempo + OTEL Collector	tracing	Helm — grafana / open-telemetry	Distributed tracing backend and vendor-neutral trace ingest
observability-sample-app	demo-app	Own repository, own GHCR image	Real instrumented workload proving the pipeline end-to-end
node-exporter	kube-system	Subchart of kube-prometheus-stack	Host-level metrics; requires host access incompatible with restricted namespaces

3.3 Key Architectural Decisions

Everything through ArgoCD, with exactly one exception

The only manifests ever applied directly with `kubectl` are ArgoCD's own installation and a single root Application that bootstraps the App-of-Apps pattern. Every other object in the cluster — namespaces, RBAC, NetworkPolicies, the entire metrics/logs/traces stack, dashboards, alert rules, and the sample application — is a child Application reconciled automatically from this repository. This is elaborated in Section 5.

Namespace-per-concern multi-tenancy

Four tenant namespaces — `observability`, `logging`, `tracing`, `demo-app` — each carry their own ResourceQuota, LimitRange, restricted Pod Security Standard labels, and default-deny NetworkPolicy with explicit allow rules. `kube-system` deliberately opts out of the restricted policy because it hosts the two DaemonSets (node-exporter, Promtail) that require genuine host access — a constraint identified and documented before either component was deployed, not discovered as a surprise mid-build.

Everything version-pinned, nothing left floating

The Kubernetes version, every Helm chart version, and every container image tag are pinned to an exact version — never `latest` or a floating channel. The sample application goes further: its deployment references an immutable commit-SHA image tag, so "what is running in the cluster" always maps to one exact, inspectable source commit.

A CNI that actually enforces policy

Minikube's default CNI does not enforce NetworkPolicy at all — this was proven with a live test where traffic crossed a default-deny policy that should have blocked it, and didn't. The cluster was switched to Calico specifically so NetworkPolicy has real teeth, matching how GKE (Dataplane V2) and EKS (Calico / VPC CNI policy support) enforce network policy in production.

4. Technology Stack and Dependencies

4.1 Platform and Tooling

Layer	Technology	Version
Container orchestration	Kubernetes (Minikube)	v1.35.1, pinned
Local cluster runtime	Minikube	v1.38.1
CNI / network policy enforcement	Calico	Minikube-managed add-on
Cluster CLI	kubectl	v1.35.3
Package manager	Helm	v4.1.4
GitOps controller	ArgoCD (server + CLI)	v3.4.5
Application runtime (sample app)	.NET SDK	10.0.301

4.2 Observability Stack — Helm Chart Dependencies

Chart	Repository	Pinned Version	Delivers
kube-prometheus-stack	prometheus-community	87.15.2	Prometheus, Alertmanager, Grafana, kube-state-metrics, Prometheus Operator, node-exporter subchart
loki	grafana	7.0.0	Log aggregation backend (SingleBinary mode)
promtail	grafana	6.17.1	Node-level log shipping DaemonSet
tempo	grafana	1.24.4	Distributed tracing backend (single-binary mode)
opentelemetry-collector	open-telemetry	0.165.0	Vendor-neutral OTLP trace ingestion gateway

4.3 Why Pinned Dependencies Matter Here

Every chart version above is locked in both the GitOps Application manifests and the two distributable Helm charts' `Chart.lock` files. This is a deliberate governance choice: a chart maintainer publishing a new default (as happened during this build, when a newer OpenTelemetry Collector chart version silently dropped its default container image) cannot change what is running in this platform without a reviewed, version-controlled commit. For an enterprise environment, this converts "a third-party dependency changed under us" from a production incident into a routine, reviewable pull request.

5. GitOps Delivery Model — ArgoCD

5.1 Why GitOps

Traditional Kubernetes operations rely on operators running `kubectl apply` or `helm install` commands directly against a cluster. This is fast to start with and difficult to govern at scale: there is no single place to see what changed, when, why, or who approved it, and the live cluster state can silently drift from whatever was last documented. GitOps inverts this relationship — **the cluster continuously converges toward whatever is declared in a Git repository**, and a controller (here, ArgoCD) is responsible for making that true, not a human running commands.

5.2 The App-of-Apps Pattern

This platform uses ArgoCD's "App-of-Apps" pattern: a single root Application (`bootstrap/root-app.yaml`) watches one directory in this repository (`argocd-apps/`) and turns every Application manifest found there into a live, independently-reconciled child Application. Adding a new platform component is therefore: write the desired-state manifests, drop one Application file into `argocd-apps/`, commit, and push — ArgoCD does the rest, with no new tooling and no change to the bootstrap process.

Applications under management	Count
Platform infrastructure (namespaces, RBAC, NetworkPolicy, security, cert-manager)	5
Observability stack (metrics, logging, promtail, tracing, otel-collector)	5
Content as code (dashboards, alert rules)	2
Workload (sample application)	1
Root bootstrap Application	1
Total ArgoCD-managed Applications	14

5.3 How This Improves Kubernetes Maintainability

Traditional operations	With ArgoCD (this platform)
Change history lives in shell history, tickets, or nowhere	Change history is the Git commit log — every change is a diff, with an author and a message
Rollback means remembering the previous command	Rollback is <code>git revert</code> ; ArgoCD reconciles the cluster back automatically
Configuration drift accumulates silently between deploys	Drift is corrected automatically — <code>selfHeal: true</code> reverts any live edit that wasn't pushed to Git, usually within seconds
"What's actually running?" requires querying the live cluster	What's running is provably what's in the pinned Git revision ArgoCD reports as Synced
Multi-cluster consistency requires re-running scripts per cluster, prone to skew	The same repository can bootstrap any number of clusters identically — deployment is declarative, not procedural
Access to change production requires broad <code>kubectl</code> credentials	Changing production requires only Git write access; cluster-admin credentials can be far more tightly held
Detecting an unauthorized or accidental manual change is hard	ArgoCD's diff view and self-heal make any out-of-band change immediately visible and automatically reverted

5.4 Self-Healing in Practice

Every Application in this platform runs with `automated: {prune: true, selfHeal: true}`. In practice this means any direct `kubectl` or `helm` edit made against the live cluster — including edits made for urgent troubleshooting — is treated as drift and reverted to match the last commit pushed to Git, typically within three to five seconds. This was observed repeatedly during hands-on development and is **correct GitOps behavior, not a defect**: Git is authoritative by design. The documented technique for safely validating a fix live before committing it is to temporarily pause that one Application's sync policy (`argocd app set <name> --sync-policy none`), verify the fix, push it, then restore automated sync. This same behavior is what makes the platform resilient against configuration drift in production: a well-intentioned but undocumented manual fix cannot quietly become permanent, hidden state.

Governance benefit: because RBAC in this platform deliberately excludes namespace-admin personas from editing RBAC, ResourceQuota, LimitRange, and NetworkPolicy objects directly (Section 7), and those same objects are ArgoCD-managed with self-heal, governance configuration in this cluster can only change through a reviewed Git commit — not through elevated runtime access, and not by accident.

6. Monitoring and Observability Configuration

The platform implements all three observability pillars — metrics, logs, and traces — as independently scalable components that are unified in a single Grafana pane, and proves the unification with a real request rather than a synthetic demonstration.

6.1 Metrics — Prometheus

Deployed via the `kube-prometheus-stack` chart with **Prometheus running 2 replicas** for availability, backed by persistent storage. Metrics sources include:

- Kubernetes control-plane and workload metrics via kube-state-metrics
- Node-level host metrics via node-exporter (DaemonSet, one per node)
- Application metrics scraped directly from the sample workload's `/metrics` endpoint — no push gateway in the path
- Prometheus Operator CRDs (`ServiceMonitor`, `PrometheusRule`) used throughout, so scrape targets and alert rules are themselves Kubernetes objects managed by ArgoCD

At last validation, all active scrape targets reported healthy (`21/21 up`). Three control-plane targets (kube-scheduler, kube-controller-manager, kube-etcd) are intentionally excluded on this Minikube reference environment, as Minikube hardcodes those static pods to bind to localhost only — a Minikube-specific limitation, not a design gap; on GKE/EKS these are cloud-managed and exposed differently.

6.2 Visualization — Grafana

Grafana is provisioned with persistent storage and pre-wired datasources for Prometheus, Loki, and Tempo, so an operator can pivot between metrics, logs, and traces without manually configuring a single connection. Dashboards are **defined as code** — raw dashboard JSON committed to this repository, wrapped into a ConfigMap via Kustomize, and picked up automatically by Grafana's sidecar watcher. A platform-overview dashboard and a sample-application dashboard are included; the platform overview generalizes to any workload, while the sample-app dashboard demonstrates request-rate, error-rate, and latency panels backed by real traffic.

6.3 Logging — Loki and Promtail

Loki runs in SingleBinary mode — the appropriate topology at this scale, avoiding the operational overhead of a multi-component read/write/backend split that would only add complexity without a corresponding benefit. Promtail runs as a DaemonSet in `kube-system` (it requires host path access incompatible with the platform's restricted namespaces) and tails every container's stdout across the entire cluster. At last validation, Loki held ingested log streams from all active application and platform namespaces — `every namespace, confirmed via Loki's label API`, not assumed from configuration alone.

6.4 Distributed Tracing — OpenTelemetry and Tempo

Application traces are exported over OTLP to a dedicated OpenTelemetry Collector, which forwards them to Tempo (single-binary mode, persistent storage). A dedicated collector sits in front of Tempo by design — even though Tempo itself accepts OTLP directly — so instrumented applications depend on a vendor-

neutral ingestion point rather than being coupled to Tempo specifically, preserving the option to redirect traces to a different backend later without touching application code.

6.5 Alerting — Alertmanager and SLO Rules

Alertmanager runs a **3-replica cluster** — deliberately an odd number for real gossip-protocol quorum, since two replicas provide no genuine high-availability advantage over one. Alert rules are authored as `PrometheusRule` custom resources, version-controlled alongside everything else, and organized into three groups:

- **Node health** — high CPU, high memory, node-not-ready conditions
- **Pod crash-loop detection** — cluster-wide, workload-agnostic
- **Application SLO** — a burn-rate error-rate alert pattern for the sample application, intentionally a simplified single-window model rather than the full multi-window Google SRE workbook approach, documented as such so the trade-off is explicit rather than implied.

These rules were validated with **two real fired-and-resolved alerts**, not simulated conditions: a deliberately induced failing endpoint drove the SLO error-rate alert from pending to firing to resolved, confirmed live in Alertmanager; and the crash-loop alert correctly caught a genuine tracing-backend memory issue during load testing, which was then diagnosed and fixed — evidence the alerting pipeline detects real problems, not only lab conditions.

6.6 Cross-Pillar Correlation

The platform's central observability claim — that metrics, logs, and traces work together, not merely alongside each other — was proven directly: a single real request's trace ID was located in both Tempo (as the actual multi-span distributed trace) and Loki (as a structured log line carrying the identical trace ID field), while the corresponding business metric incremented correctly in Prometheus for the same request. One trace ID, one real request, present and correlated across all three pillars simultaneously.

6.7 The Instrumented Reference Workload

An ASP.NET Core (.NET 10) sample application was purpose-built to prove the pipeline rather than generate synthetic load. A single API call (`GET /api/orders/{id}/enriched`) triggers a genuine internal HTTP call to a second endpoint, producing a real three-span distributed trace rather than a single isolated span. The application exports:

- **Metrics** — ASP.NET Core auto-instrumentation plus custom business-event counters, scraped directly by Prometheus
- **Traces** — OTLP export to the in-cluster collector, endpoint configured via environment variable, never hardcoded
- **Logs** — structured JSON to stdout, enriched with the active trace and span ID, tailed by the same cluster-wide Promtail pipeline as every other workload — deliberately not a second, parallel logging path

7. Security and Governance

7.1 Role-Based Access Control

Two personas are defined per tenant namespace, bound to **Kubernetes Groups** rather than individual ServiceAccounts, so they can be wired directly into a real identity provider's group claims later without any change to these manifests:

- `sre-viewer` — read-only access, explicitly cannot read Secrets
- `observability-admin` — full CRUD on workload resources, but deliberately excluded from RBAC objects, ResourceQuota, LimitRange, and NetworkPolicy — governance configuration remains GitOps-only, not self-service even for a namespace administrator

7.2 Network Segmentation

Every tenant namespace enforces a default-deny-all NetworkPolicy with explicit, narrowly scoped allow rules layered on top — traffic is denied by default and permitted only where a documented need exists (for example, Grafana querying Prometheus, or Promtail pushing logs into Loki from a different namespace). This is enforced at the CNI layer via Calico, confirmed with a live positive-and-negative connectivity test rather than assumed from the policy YAML alone.

7.3 Pod Security Standards

The `restricted` Pod Security Standard is enforced via namespace labels on all four tenant namespaces — no privilege escalation, no root containers, mandatory seccomp profiles, dropped Linux capabilities. Components that structurally require host-level access (node-exporter, Promtail) are namespaced into `kube-system` instead of weakening the tenant namespaces' policy, a conflict identified and documented before either component was deployed.

7.4 Certificate Management

cert-manager is deployed with a self-signed `ClusterIssuer` on the Minikube reference environment, proving the certificate-issuance pattern end-to-end. The GKE and EKS documentation (Section 9) specifies the cloud-managed equivalents — Google Certificate Manager with Workload Identity, and AWS Certificate Manager with IRSA — as direct drop-in replacements for production use.

8. Phased Implementation Process

The platform was delivered in ten sequential, independently validated phases. Each phase's output was verified against the live cluster before the next began — the table below is a condensed record; the full hands-on log, including every real defect found and how it was diagnosed and fixed, is maintained in the project's technical documentation set.

Phase	Delivered	Notable finding
1. Cluster provisioning	Sized, version-pinned Minikube cluster with dynamic storage and Calico CNI	Default CNI does not enforce NetworkPolicy at all — proven live, not assumed
2. GitOps foundation	ArgoCD installed; root App-of-Apps bootstrapped	Large CRDs require server-side apply — a known ArgoCD installation constraint
3. Namespace & multi-tenancy	Four tenant namespaces with quotas and limits	Initial sizing needed two real corrections once live workloads and sync hooks landed
4. Security & RBAC baseline	RBAC, NetworkPolicy, Pod Security Standards, cert-manager	Wildcard RBAC initially over-granted secret access — corrected to explicit per-resource rules
5. Metrics stack	Prometheus, Alertmanager, Grafana (kube-prometheus-stack)	Four distinct chart-default gaps found only by deploying for real (security context, resource sizing, host-access conflicts)
6. Logging stack	Loki (SingleBinary) + Promtail	Chart topology default conflicted with SingleBinary mode; corrected explicitly
7. Tracing stack	Tempo + OpenTelemetry Collector	A recent collector chart version dropped its default image — caught immediately by a failed deploy, not silently
8. Reference workload	Instrumented ASP.NET Core sample application, own CI/CD to GHCR	Multi-span trace design proves distributed propagation, not just span existence
9. Dashboards, alerts & SLOs	Dashboards and alert rules as code	Alerting caught a genuine production-shaped defect (Tempo memory pressure) during its own validation traffic
10. End-to-end validation	26-check automated sign-off gate	Clean run required as a hard gate before any documentation phase could begin

Two further phases packaged the platform for distribution: Helm chart authoring and validation for GKE and EKS (Section 9), and final reference documentation. A reproducible, unattended rebuild script (`scripts/complete-setup.sh`) chains all of the above end to end for future re-provisioning.

9. Multi-Cloud Packaging — GKE and EKS

The architecture validated on Minikube is packaged as two structurally identical Helm charts — `observability-gke` and `observability-eks` — sharing the same four pinned subchart dependencies and the same dashboard/alert-rule content. Only genuinely cloud-specific values differ.

Value	GKE chart	EKS chart
Persistent volume storage class	premium-rwo	gp3
Grafana Ingress class	gce	alb
Workload identity federation	Workload Identity (GCP service account annotation)	IRSA (IAM role ARN annotation)
Object storage overlay (opt-in)	Google Cloud Storage backend	Amazon S3 backend

Validation Performed

Neither chart has a real GKE or EKS cluster in this project's scope, so validation used three progressively stronger layers:

1. `helm lint` — both charts pass cleanly
2. **Schema validation** — full rendered output of both charts, with and without their object-storage overlay, validated against Kubernetes and Prometheus-Operator CRD schemas:

130/130 resources valid, 4/4 render variants
3. **Live API server dry-run** — `helm install --dry-run=server` against the real Minikube API server, which validates against actual installed CRD definitions. This step found and fixed a genuine defect: both charts' own templates used a separate namespace value inconsistently from every subchart, which could have silently split a real deployment across two namespaces under a non-default install namespace. Corrected and re-verified.

10. Quality Assurance and Validation

An automated, repeatable validation script (`scripts/validate.sh`) serves as the platform's sign-off gate, checking seven categories end-to-end against the live cluster rather than against static configuration:

Category	What is checked
ArgoCD state	All 14 Applications report Synced and Healthy
Metrics	Every Prometheus scrape target reports healthy
Logs	Loki holds ingested streams from all expected namespaces
Traces	Tempo returns real recent spans for the reference workload
Dashboards	Grafana renders the provisioned dashboard with its expected panels
RBAC	Least-privilege behavior verified by impersonation (can/cannot, not configuration inspection)
NetworkPolicy	Default-deny policy structurally present in every tenant namespace

All data-plane checks run from inside the cluster (via a pod already on the cluster network) rather than through an operator's local port-forward tunnel — a more honest test of what actually matters for production (in-cluster service reachability), and immune to the connectivity flakiness of long-lived local tunnels. A clean run reports all checks passing before any downstream documentation or packaging phase is considered complete.

11. Operational Lessons and Runbook

Self-heal reverts unpushed live edits — by design

Any direct cluster edit not yet committed to Git is treated as drift and reverted, typically within seconds. This is the correct, intended GitOps behavior, and the documented workaround for validating a fix live before committing is to temporarily pause the specific Application's sync policy, never to disable self-heal platform-wide.

Grafana's admin credential can desync from its own Secret once persisted

The admin password environment variable only seeds a brand-new database on first boot; once persistent storage is enabled, later credential rotations do not automatically propagate to the already-existing admin account. The durable fix is resetting the password through Grafana's own CLI inside the running pod, which updates the persisted database without disturbing dashboards or datasources.

Third-party Helm chart defaults are a starting point, not a compliance guarantee

Every Helm-deployed component in this platform required at least one real, live-deployment correction that reading the chart's documentation alone would not have surfaced — a missing security field, a resource default colliding with namespace limits, a value nested at the wrong key path in an upstream chart's schema (found during this engagement's own live cross-check and corrected). The pattern that reliably catches these: inspect a chart's actual rendered defaults before writing values, deploy against a real cluster, read the actual error, fix it, and re-verify — rather than trusting documentation or a clean template render alone.

Reproducibility from a clean slate

The entire platform — cluster, GitOps controller, and every managed component — can be rebuilt unattended from a single script following a Minikube teardown, provided the Git repository already holds the desired state. This was a deliberate design goal, not an afterthought: infrastructure that can only be recreated by someone who remembers the original manual steps is not truly GitOps-managed.

12. Conclusion and Recommendations

This engagement delivered a working, provably correlated observability platform — not a proof-of-concept dashboard, but a system validated with real traffic, real fired alerts, and a real traced request — governed entirely through GitOps, hardened with enterprise-baseline security controls, and packaged for deployment on two major public clouds without architectural change. The 26-check automated validation gate and the reproducible setup script together mean this platform's correctness is a repeatable test result, not a point-in-time claim.

Recommended Next Steps for Production Adoption

1. **Identity federation** — connect the existing RBAC Group bindings to a real identity provider (OIDC/SAML), which requires no change to the RBAC manifests themselves, only IdP configuration.
2. **Durable object storage for logs and traces** — enable the already-built GCS/S3 overlay for Loki and Tempo before retention needs exceed local persistent volume capacity.
3. **Cloud-managed certificates** — replace the self-signed issuer with Google Certificate Manager or AWS Certificate Manager plus Workload Identity/IRSA, both already documented as direct replacements.
4. **Continuous integration for this repository** — add automated `helm lint` /schema validation on every pull request, extending the manual validation already performed to a standing CI gate.
5. **Formal alert routing** — wire Alertmanager's receivers to real paging/notification channels; the alert rules and severities are already defined and proven to fire correctly.

13. Appendix

13.1 Repository Map

Kubernetes-grafana-prometheus/	platform/GitOps content + Helm charts + docs
├ bootstrap/	ArgoCD's own install + the App-of-Apps root Application
├ argocd-apps/	14 child Applications, one per component
├ manifests/	desired-state content every Application points at
├ charts/	observability-gke/, observability-eks/
├ scripts/	cluster + ArgoCD bootstrap, validation, complete rebuild
└ docs/	architecture, phase-by-phase record, cloud docs, chart reference
observability-sample-app/	sibling repository – application code only

13.2 Glossary

Term	Meaning in this context
GitOps	Operating model where a Git repository is the single source of truth for desired cluster state, continuously reconciled by an in-cluster controller
App-of-Apps	An ArgoCD pattern where one root Application manages a set of child Applications, enabling one bootstrap step to manage an arbitrarily large platform
Self-heal	ArgoCD behavior that automatically reverts any live cluster state that doesn't match the last-synced Git commit
OTLP	OpenTelemetry Protocol — the vendor-neutral wire format used to export traces (and optionally metrics/logs) from an application to a collector
Pod Security Standards (PSS)	Kubernetes-native policy levels (privileged, baseline, restricted) enforced via namespace labels, replacing the deprecated PodSecurityPolicy
SLO burn-rate alert	An alert that fires based on how quickly an error budget is being consumed, rather than on a single static threshold
Workload Identity / IRSA	Cloud-native mechanisms (GCP / AWS respectively) allowing a Kubernetes ServiceAccount to assume a cloud IAM identity without storing long-lived credentials

End of report. Prepared from the project's technical documentation set (architecture, phase-by-phase implementation record, cloud translation guides, and Helm chart reference) maintained in the platform repository.